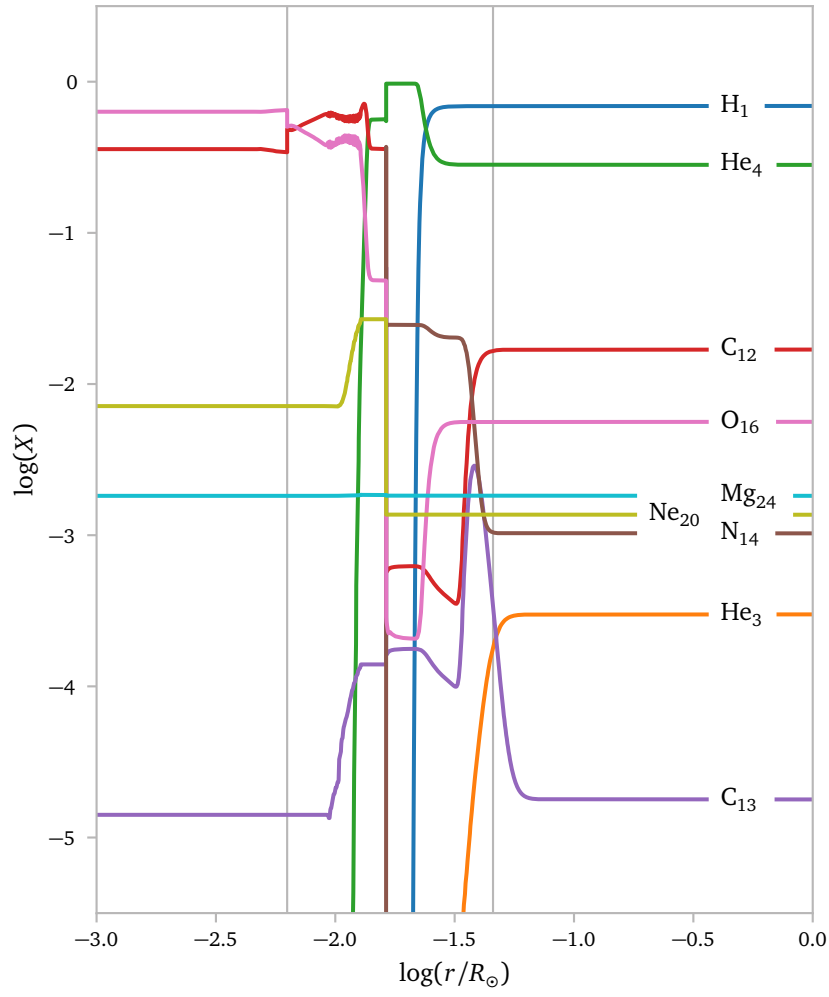


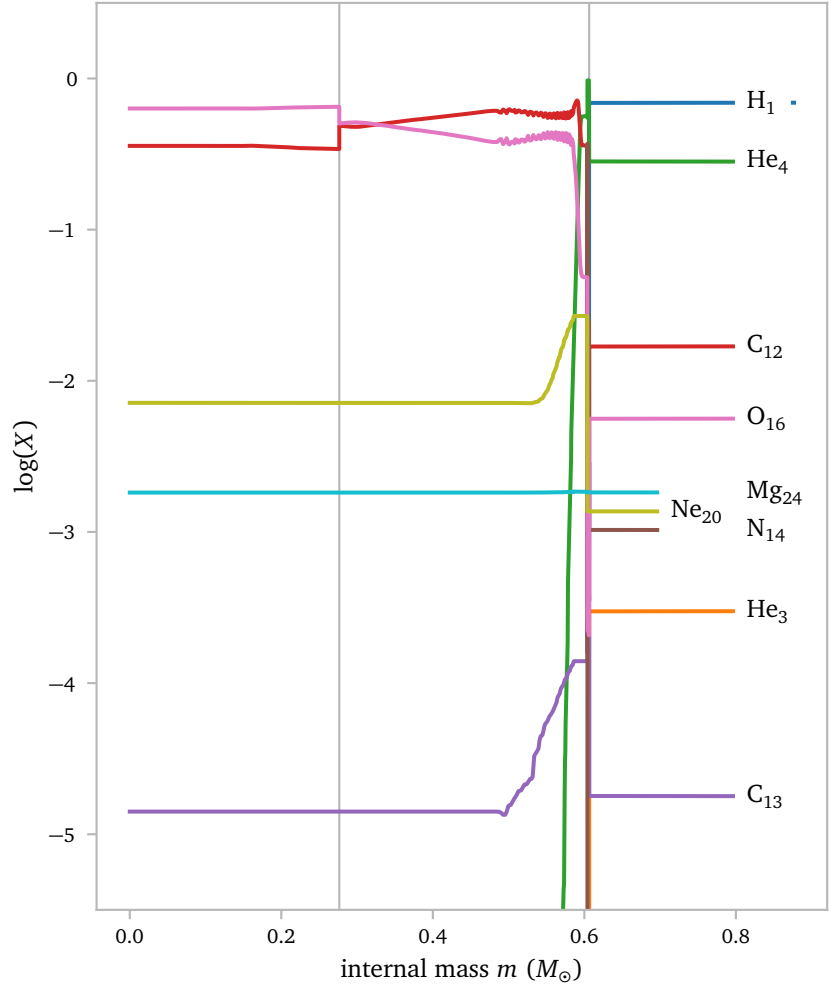


The vertical gray lines show the points where the problems in the temperature gradient start.

The first line seems so perfectly align with a small jump in C_{12} and a small drop in O_{16} .

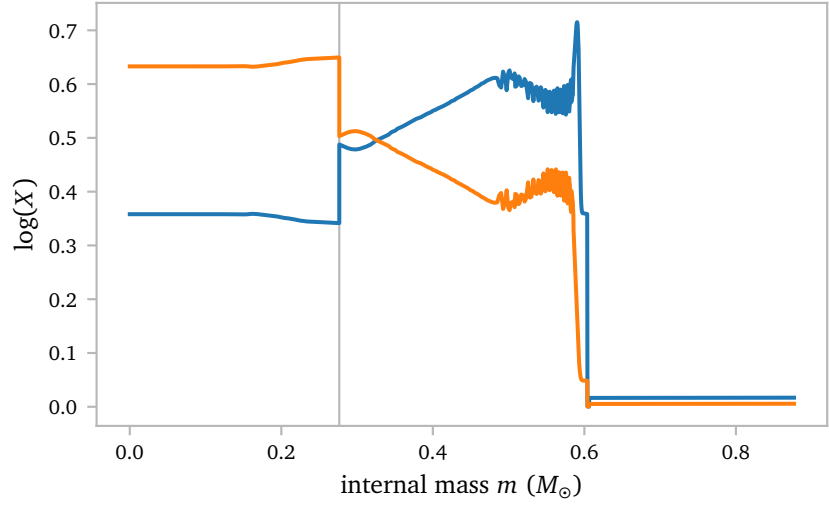
The second line does not correspond with such a jump, but I also think this is not the *real* reason the model is failing, and it is much more likely that this problematic area only exists because the first problem could not be solved.

Below, I plot the same figure but now as a function of internal mass.



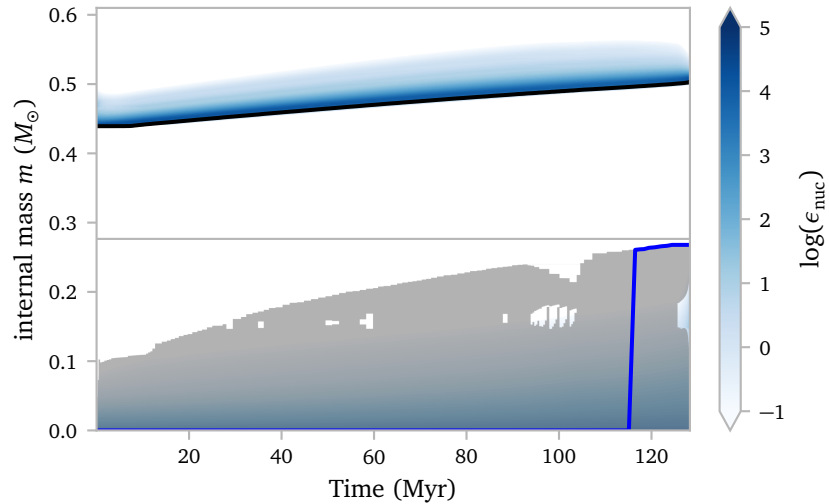
Again, the jump in C_{12} and dip in O_{16} are quite clear.

Here is a zoom:



I think the boundary between the flat interior and the sloping region outside $m \approx 0.27$ aligns with the boundary between the fully mixed region during core helium burning and the region outside of it (Straniero et al., 2003).

I think this can be seen in the Kippenhahn plot below.



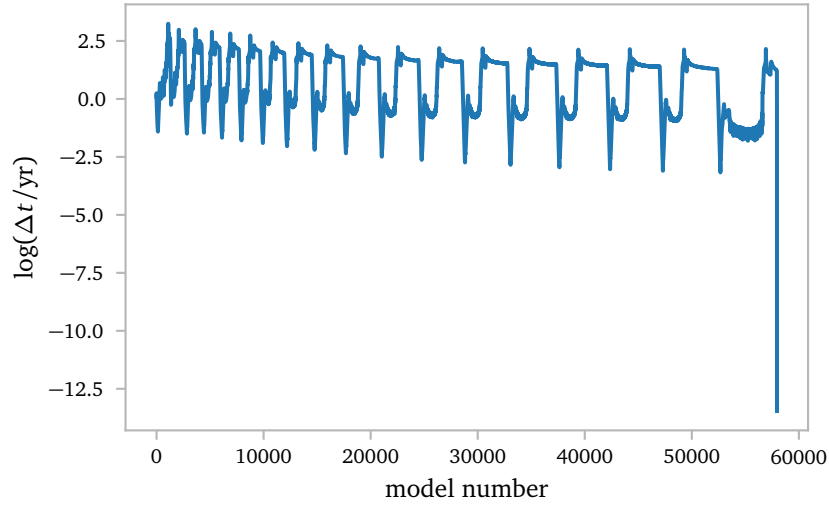
The gray regions are convective regions, while the white regions are radiative. The gray horizontal line aligns with the location of the C_{12} and O_{16} jumps and the instability.

Keep in mind that this Kippenhahn plot is for a solar metallicity model (one of the earlier ones because I did not have the needed profiles for a detailed kippenhahn plot during the CHeB-phase).

So the jumps are probably a *real physical feature*. They do however align with a discontinuity of the nuclear energy generation, entropy and opacity.

I hope that I can fix this by simply improving the mesh around this troublesome point.

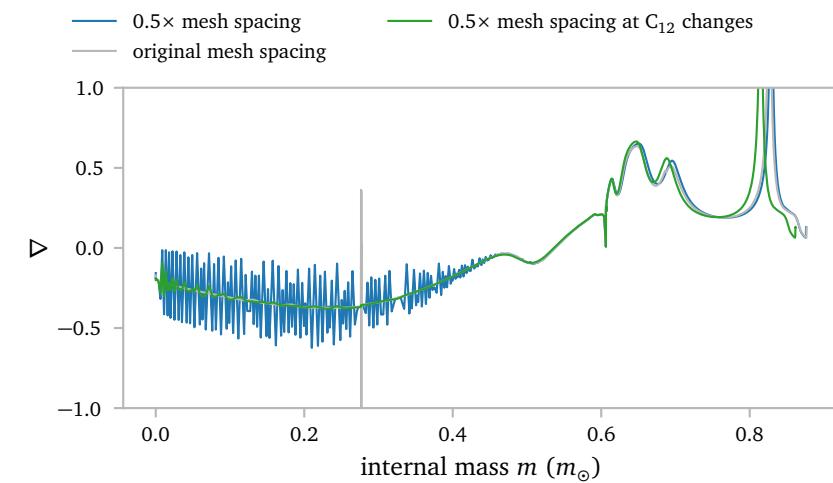
Lastly, I think the Δt plot is interesting.



The simulation completely grinds to a halt.

a.2 Increased Mesh Refinement

Increasing the mesh refinement using `mesh_delta_coeff = 0.5d0` seems to prevent the instability. The model can continue to evolve *but* the temperature gradient is generally a lot less smooth and the evolution obviously takes significantly longer for a more refined mesh.



This does not seem like a good solution.

I also tried an option for a denser mesh spacing around C_{12} abundance changes. This is probably a more efficient way to prevent the instability, but it is very difficult to find the correct settings. The green line is the best I was able to find, but this requires a lot of fine tuning and rerunning of the model and is not a good *general* solution.

a.3 Other Metallicities

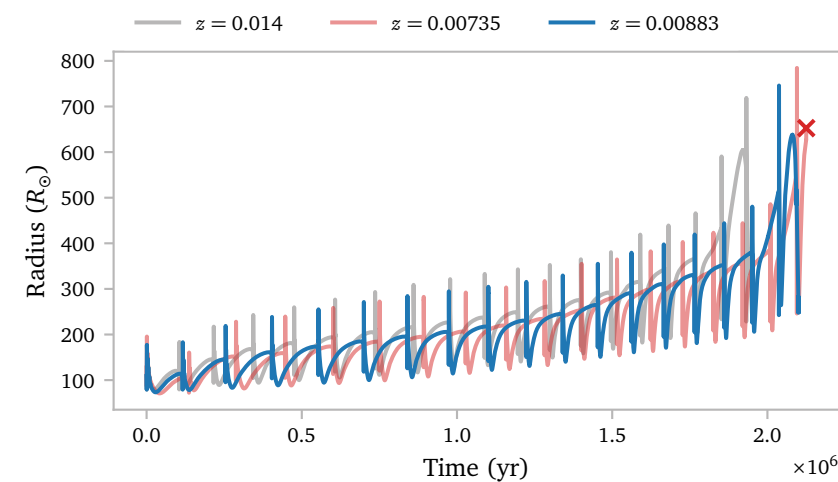
From the samples of [de Castro et al. \(2016\)](#) we can find slightly different metallicities.

median([FeI/H]) = −0.202,
weighted average([FeI/H]) = −0.222,
mean([FeI/H]) = −0.150.

median([FeII/H]) = −0.204,
weighted average([FeII/H]) = −0.222,
mean([FeII/H]) = −0.155.

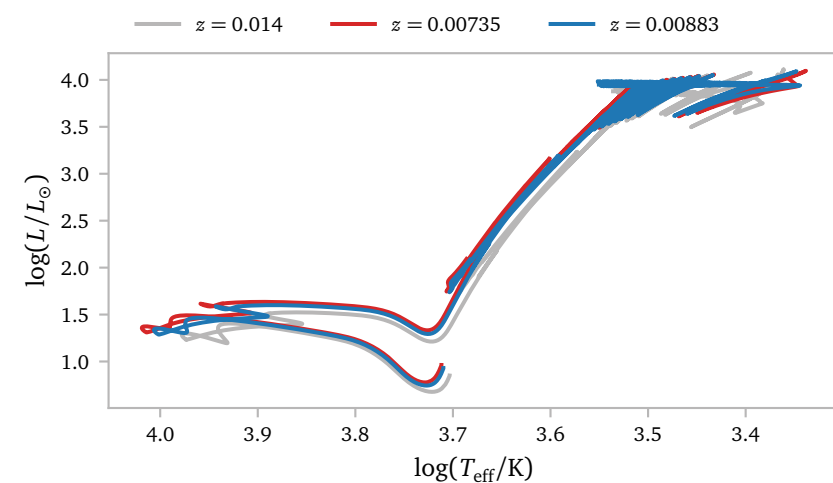
[de Castro et al. \(2016\)](#) fit a double Gaussian to the metallicity data and find a Gaussian for thin disk stars with $\mu = -0.12$, $\sigma = 0.14$ and a Gaussian for thick disk stars with $\mu = -0.49$, $\sigma = 0.09$.

I start by simulating a model with $[\text{Fe}/\text{H}] = -0.2 \rightarrow Z \approx 0.0883$.



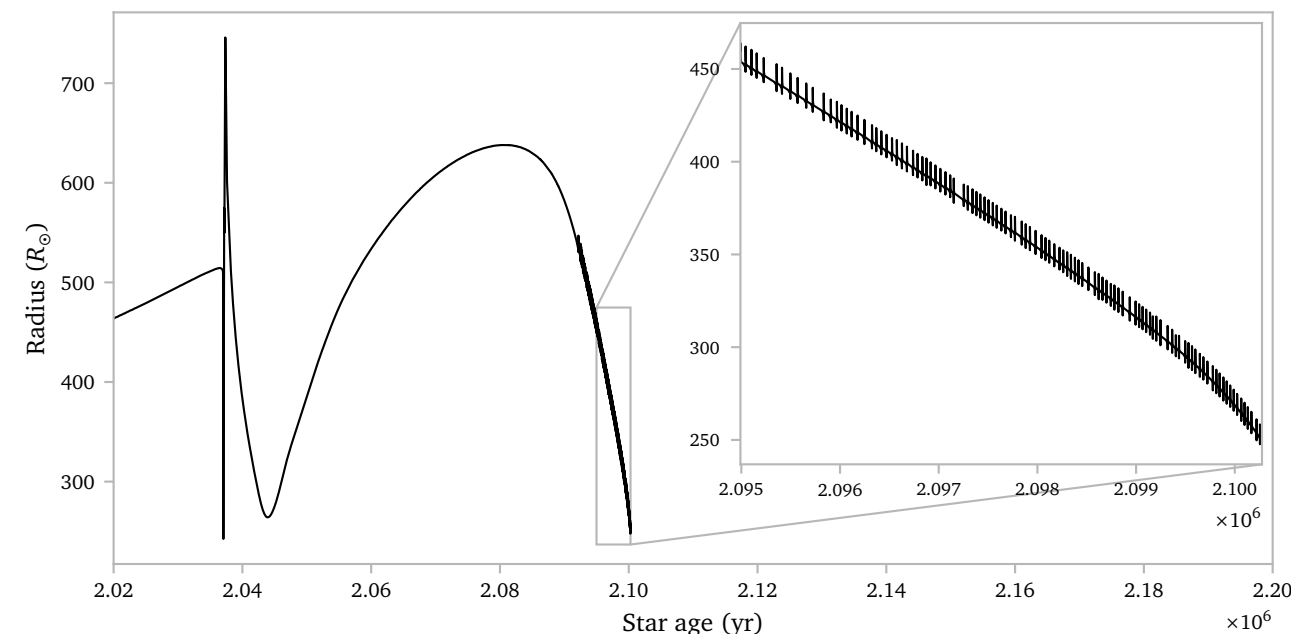
This model seems to run just fine. Its behaviour is somewhere between the solar metallicity and the more metal poor star. This all makes sense and it is nice to know that *MESA* can simulate metal-poorer stars, even though this star is not very metal poor at all.

Below, I plot the effects of the metallicity on the HR-diagram.



The main effect is the slight horizontal shift in effective temperature. This is what I would expect.

The solar metallicity model and the new model both experience radial instabilities when become post-AGB stars. I do not think this is super important, but a saved a bunch of profiles for the new star so I can actually see what is happening inside the star.



The instabilities are show as pulsating bumps in the entropy profile close to the core-boundary of the star. The instabilities are not very visible when plotting the temperature gradient or the energy generation rate.

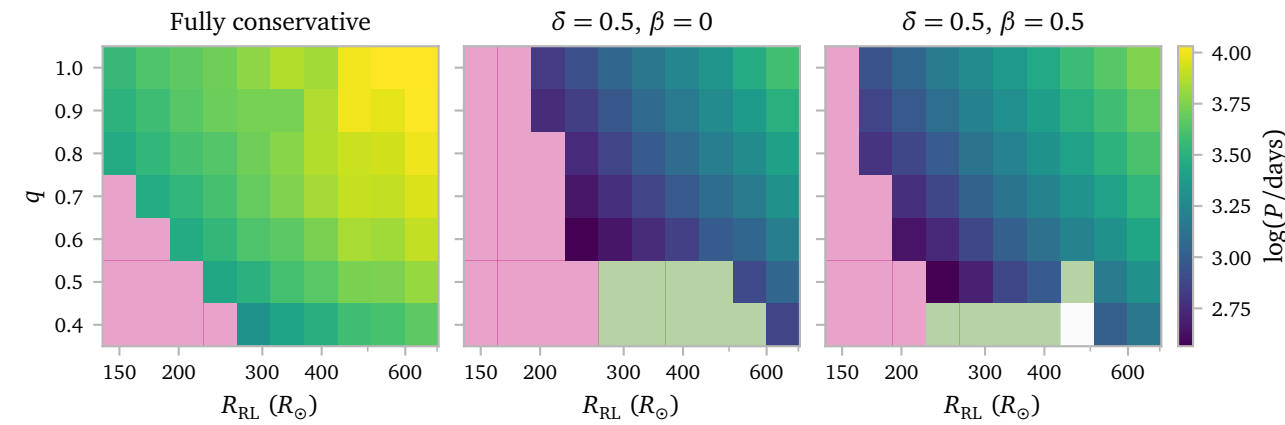
[Click here!](#)

b. Fully Non-Conservative Mass Transfer

Last week I started a grid of simulations with $\delta = 0.5$ $\gamma = 1$ and $\beta = 0.5$ such that all RLOF mass transfer would be fully non-conservative.

My main hypothesis was that this effect of β would not change the orbits much, but will have a significant effect on the final mass of the barium star, and the C/O-ratio and other abundances.

Here, I plot the final orbits.



[Click here for an interactive view!](#)

This result does not immediately make sense to me. I expected the orbits to be a bit *shorter*, not a bit *longer*. This is what I saw in my plots from last week as well.

It is a good thing that I checked this, because as it turns out, I forgot to add the angular momentum losses due to mass losses from the accretor to my custom $\dot{J}_{\text{ML}}$ prescription.

Effectively, some mass is now lost *without any* angular momentum losses. Since normally, mass ends up being transferred to a heavier companion, this costs angular momentum, but with isotropic re-emission without angular momentum losses, no angular momentum is consumed, which results in larger orbits.

I have fixed this now and a new grid is currently simulating on the cluster. This is the new $\dot{J}_{\text{ML}}$ prescription:

```
! --- mass lost from vicinity of donor --- saladino prescription
b%jdot_ml = b%mdot_system_wind(b%d_i) &
    *eta &
    *(b%separation)**2 &
    *sqrt(1 - pow2(b%eccentricity)) &
    *2*pi/b%period

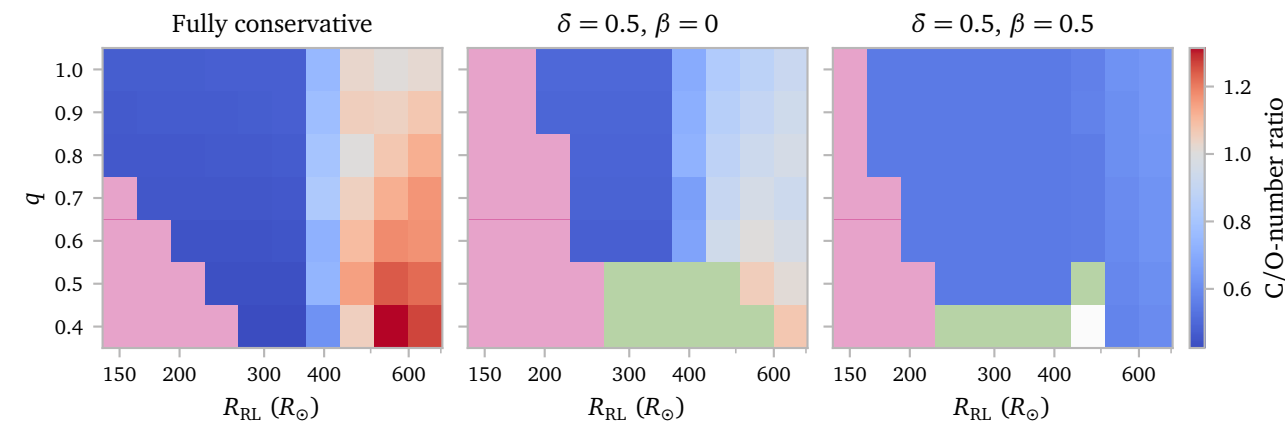
!mass lost from vicinity of accretor -- isotropic re-emission
b%jdot_ml = b%jdot_ml + (b%mdot_system_transfer(b%a_i) &
    + b%mdot_system_wind(b%a_i)) &
    * pow2(b% m(b% d_i)/(b% m(b% a_i)+b% m(b% d_i))) &
    * b% separation)*2*pi/b% period &
    * sqrt(1 - pow2(b% eccentricity))

! --- mass lost from circumbinary coplanar toroid
b%jdot_ml = b%jdot_ml + b%mdot_system_ccct * b%mass_transfer_gamma * &
    sqrt(standard_cgrav * (b% m(1) + b% m(2)) * b% separation)
```

I think the addition of this term should be sufficient for capturing the full angular momentum evolution of isotropic re-emission

Below, I plot the inferred C/O-ratio for the old grid¹. This clearly shows that the C/O-ratio does not change significantly for the model without conservative RLOF mass transfer. We can however see a small change due to winds.

¹ with the wrong $\dot{J}_{\text{ML}}$ -prescription, but it should still be representative of the abundances.



c. MESA additions

c.1 Stopping Condition

As I described last week, I want to modify my MESA models to continue running until only $0.01M_{\odot}$ envelope mass is left, *and* RLOF has ended. This requires a custom MESA stopping condition. Luckily this is quite easy. Below, I share the new `extras_binary_finish_step` code:

```
integer function extras_binary_finish_step(binary_id)
...

if (-s1%star_mdots > 3*max_Mdot_crit) then
  do_termination = .true.
  termination_code_str(t_xtra1) = "Critical mass loss rate exceeded"
end if

if ((0.9 * b% r(1)) >= b% r(1)) .and. (s1% star_mass - s1% he_core_mass < 0.01d0)) then
  do_termination = .true.
  termination_code_str(t_xtra1) = "Envelope stripped"
end if

if (b% period < 50 * secday ) then
  do_termination = .true.
  termination_code_str(t_xtra1) = "Period became smaller than 50 days"
end if

if (do_termination .eqv. .true.) then
  s1%termination_code = t_xtra1
  extras_binary_finish_step = terminate
end if
end function extras_binary_finish_step
```

I have tested this code and it works as intended.

c.2 Saving Profiles according to Envelope Mass

I also think it is interesting to save models based on the envelope mass of the system. My current implementation is pretty easily modifiable with an easy way to change the minimum envelope mass and the maximum number of profiles to save.

I simply call *this* subroutine every `extras_finish_step`.

```
subroutine save_profile_on_envelope_decrease(s)
  type (star_info), pointer, intent(inout) :: s

  integer :: i ! index for envelope mass model
  integer :: max_profiles ! index for envelope mass model
  real(dp) :: env_mass
  real(dp) :: target_mass
  real(dp) :: minimum_envelope_mass

  ! --- setup
  max_profiles = 500
  minimum_envelope_mass = 1d-3
  env_mass = s% star_mass - s% he_core_mass

  ! --- logic to determine if a profile should be saved
  i = int(s% ixtra(ix_envelope_profile_index))

  do while (i <= max_profiles)
    target_mass = s% initial_mass &
      * (minimum_envelope_mass / s% initial_mass)&
      **((i-1d0)/real(max_profiles - 1))

    if (env_mass > target_mass) exit

    s% need_to_save_profiles_now = .true.
    s% ixtra(ix_envelope_profile_index) = s% ixtra(ix_envelope_profile_index) + 1

    i = i + 1
  end do

end subroutine save_profile_on_envelope_decrease
```

It computes n values from $m_{\min}$ to $M_{\text{star},i}$ logarithmically and checks if the current envelope mass dropped below one (or more) of these values. If this is the case, I tell MESA to save a model. It seems to work fine when I tested it quickly.

d. AGB Nucleosynthesis

References

- de Castro, D. B., Pereira, C. B., Roig, F., et al. (2016) *Chemical abundances and kinematics of barium stars*. [MNRAS459, 4299](#).
- Escorza, A. & De Rosa, R. J. (2023) *Barium and related stars, and their white-dwarf companions. III. The masses of the white dwarfs*. [A&A671, A97](#).
- Jorissen, A., Boffin, H. M. J., Karinkuzhi, D., et al. (2019) *Barium and related stars, and their white-dwarf companions. I. Giant stars*. [A&A626, A127](#).
- Straniero, O., Domínguez, I., Imbriani, G., & Piersanti, L. (2003) *The Chemical Composition of White Dwarfs as a Test of Convective Efficiency during Core Helium Burning*. [ApJ583, 878](#).

A Additional Figures

